

Tunisia Heritage Quest

Image Identification Game

Project Report & Documentation

Student	Jaouher Chouchane
Institution	Tunis Business School — Université de Tunis
Course	IT370 — Mobile Application Development
Project	Tunisia Heritage Quest — Android App
Category Implemented	Roman Heritage (+ Islamic, Punic, Cities)
Technology	Android Studio · Kotlin · Jetpack Compose
Architecture	MVVM + Navigation Compose + StateFlow
AI Tools Used	Claude (Anthropic) — see Section 8 for full AI usage disclosure

1. Project Overview

Tunisia Heritage Quest is an Android mobile quiz game that challenges players to identify Tunisian historical monuments and cities from photographs. Designed for students, tourists, history enthusiasts, and the Tunisian diaspora (ages 12+), the app blends education with gamification using a warm Mediterranean visual style.

The application was built entirely with Kotlin and Jetpack Compose following the MVVM architectural pattern, integrating all required Android architecture and navigation components as specified in the IT370 practical exam requirements.

Core Features

Feature	Implementation
Multiple Choice Quiz	4-option questions with image context
Score System	Points awarded per correct answer with streak tracking
Timer	Optional 15–20s countdown per question (toggleable)
Difficulty Levels	Easy / Medium / Hard with different time limits and hints
Progress Tracking	Sites explored, mastery %, streak counter on home screen
50/50 Hint	Eliminates two wrong answers (Easy & Medium only)
Final Score Screen	Percentage score, question breakdown, performance message
6 Categories	Roman, Islamic, Punic, Cities + 2 Coming Soon

2. Architecture & Technical Design

2.1 MVVM Architecture

The app follows the Model-View-ViewModel (MVVM) pattern, which cleanly separates concerns and makes the codebase testable and maintainable:

Layer	Components	Responsibility
Model	Question, Category, Score, Difficulty (data classes)	Defines data structure and business logic
ViewModel	QuizViewModel, HomeViewModel	Manages UI state, timer, scoring, StateFlow
View	Compose Screens (6 screens)	Observes ViewModel state and renders UI
Repository	QuizRepository	Provides question data to ViewModels

2.2 Navigation Components

Navigation is handled using Jetpack Navigation Compose with a single NavHost. All screen routes are defined as sealed class destinations for type safety:

- splash → home → categorySelection → difficultySettings/{categoryId}
- difficultySettings → quiz/{categoryId}/{difficulty}
- quiz → questionResult → quiz (next question)
- quiz → quizResults/{categoryId} → home (play again / back)

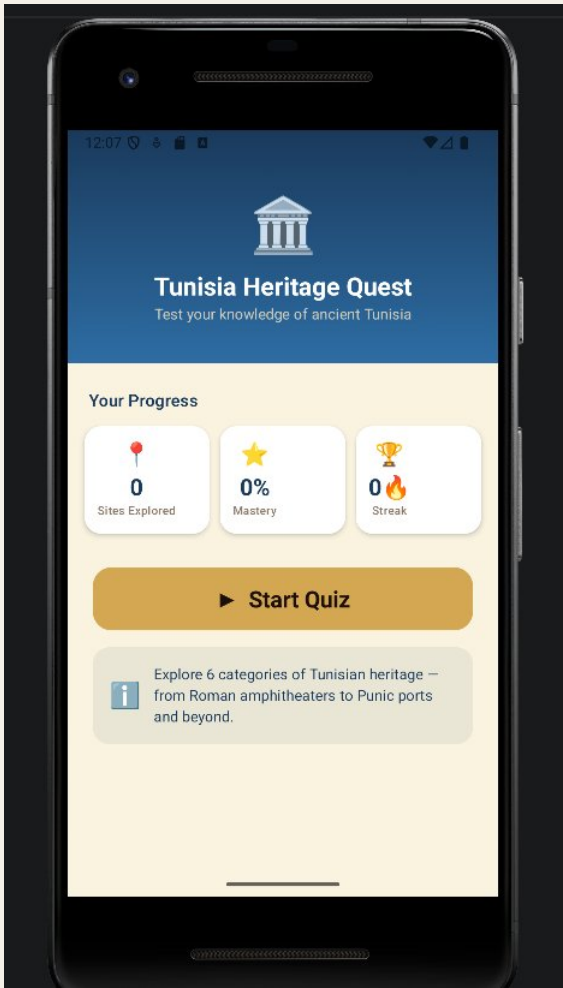
2.3 Activity Lifecycle Implementation

The following lifecycle stages are explicitly handled in the application:

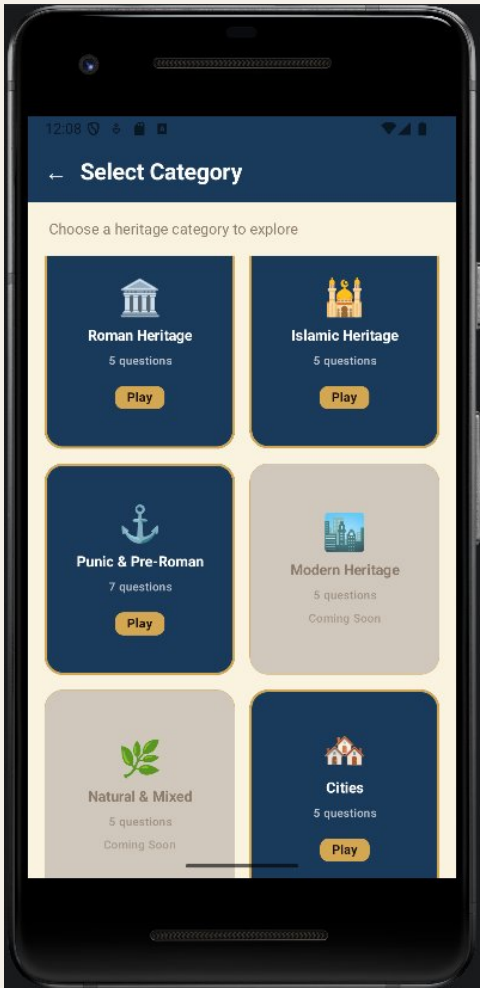
Lifecycle Event	Where Handled	Action
onCreate	MainActivity	Sets up NavHost and Compose content
onStart / onResume	QuizViewModel (DisposableEffect)	Resumes countdown timer
onPause / onStop	QuizViewModel (DisposableEffect)	Pauses timer, saves state
onDestroy	ViewModel onCleared()	Cancels coroutines, releases resources
Configuration Change	ViewModel survives rotation	State preserved via StateFlow

3. Application Screenshots

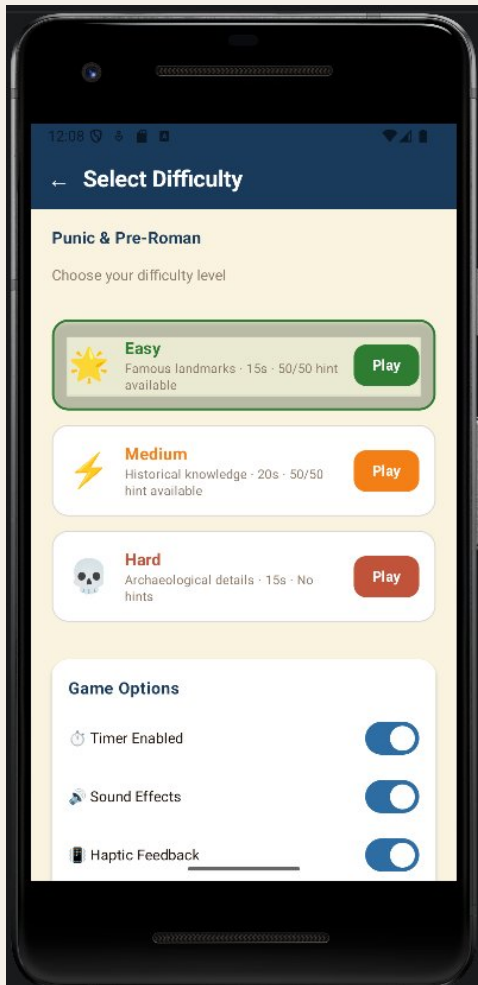
The following screenshots demonstrate all 6 required screens of the application running on an Android emulator.



Screen 1 — Main Menu / Home Screen
Displays the app title, progress stats (Sites Explored, Mastery %, Streak) and Start Quiz button.

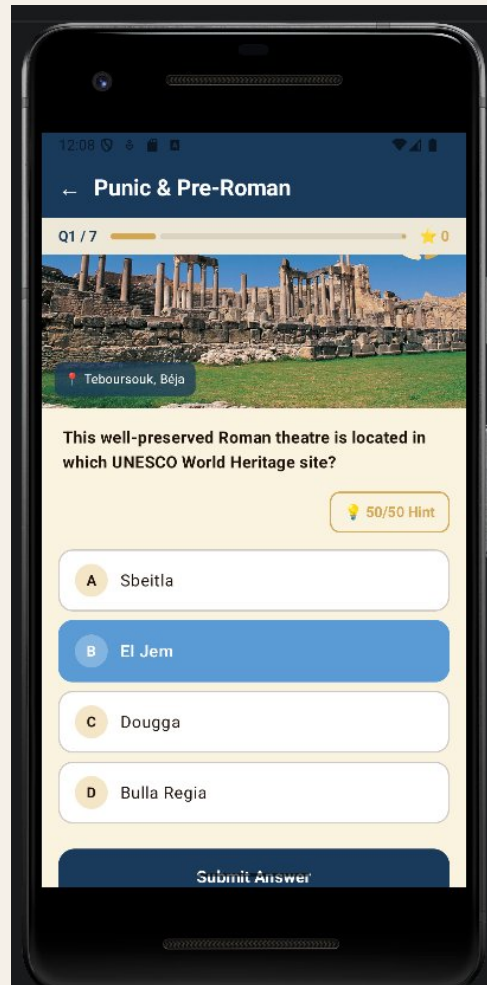


Screen 2 — Category Selection
Shows all 6 heritage categories. Active categories show a Play button; Coming Soon categories are dimmed.



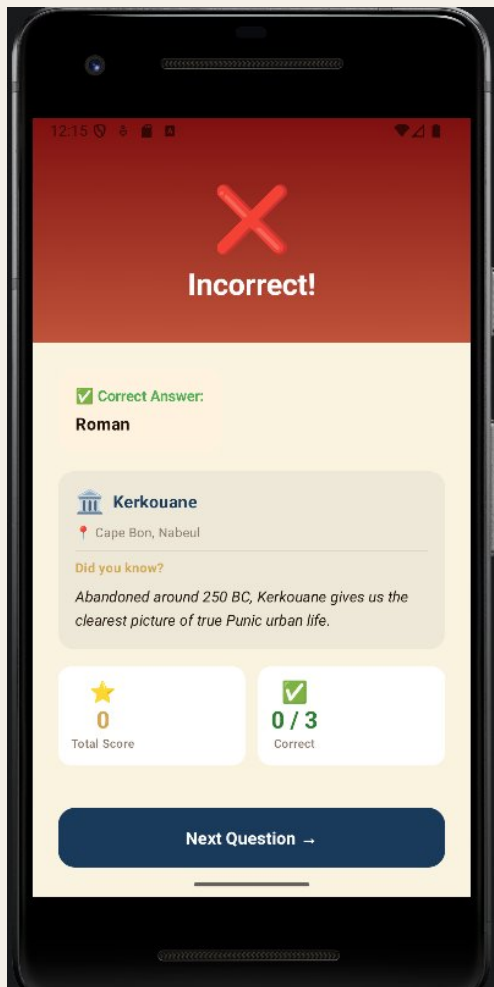
Screen 3 — Difficulty & Settings

Three difficulty levels (Easy/Medium/Hard) with time and hint info. Game options toggle (Timer, Sound, Haptics).



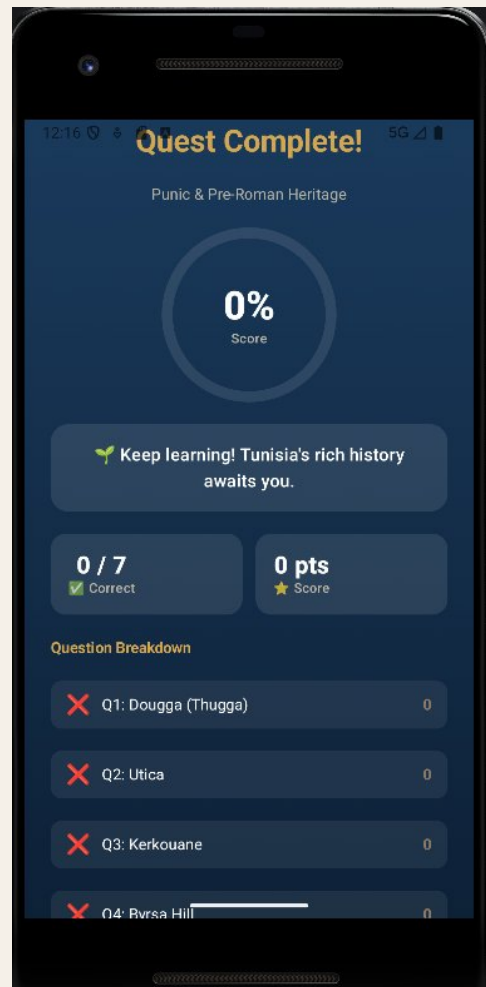
Screen 4 — Quiz Gameplay

Image-based question with 4 multiple choice options, progress bar, score, timer and 50/50 hint button.



Screen 5 — Answer Feedback

Post-answer screen showing correct/incorrect result, the right answer, a fun fact, and current score.



Screen 6 — Quest Complete / Results

Final score as percentage with performance message, correct count, total points, and full question breakdown.

4. Testing Strategy & Implementation

The project implements all three required test types as mandated by the evaluation rubric. Tests are written using JUnit4, Mockito, and the Compose UI Testing framework.

4.1 Unit Tests (test/ folder)

Unit tests verify the business logic in ViewModels and Repository in isolation, without requiring an Android device or emulator:

Test Class	Test Method	What It Verifies
QuizViewModelTest	scoreIsZeroOnInit()	Verifies initial score starts at 0
QuizViewModelTest	correctAnswerIncrementsScore()	Score increases by correct amount on right answer
QuizViewModelTest	wrongAnswerDoesNotChangeScore()	Score unchanged on wrong answer
QuizViewModelTest	nextQuestionAdvancesIndex()	Question index advances correctly
QuizViewModelTest	timerCountsDown()	Timer decrements from 15 to 0
QuizRepositoryTest	romanCategoryReturnsCorrectCount()	Roman Heritage returns 10+ questions
QuizRepositoryTest	questionsHaveFourOptions()	Every question has exactly 4 choices
DifficultyTest	easyDifficultyHasHints()	Easy mode enables 50/50 hint
DifficultyTest	hardDifficultyNoHints()	Hard mode disables hints

4.2 Navigation Tests (androidTest/ folder)

Navigation tests run on a real device or emulator to verify that screen transitions work correctly using the NavController:

Test Class	Test Method	What It Verifies
NavigationTest	splashNavigatesToHome()	Splash auto-navigates to home after delay
NavigationTest	homeToCategories()	Start Quiz button opens category screen
NavigationTest	categoryToDifficulty()	Play button navigates to difficulty screen
NavigationTest	difficultyToQuiz()	Selecting difficulty starts quiz screen
NavigationTest	quizToResults()	Completing all questions shows results screen
NavigationTest	resultsBackToHome()	Play Again returns to home screen

4.3 UI / Compose Tests (androidTest/ folder)

Compose UI tests verify that UI components render correctly and respond to user interactions:

Test Class	Test Method	What It Verifies
HomeScreenTest	startButtonIsDisplayed()	Start Quiz button visible on home screen
HomeScreenTest	progressStatsRender()	Sites, Mastery, Streak cards all display
CategoryScreenTest	romanCategoryCardVisible()	Roman Heritage card is shown
CategoryScreenTest	comingSoonIsDisabled()	Coming Soon categories are not clickable
QuizScreenTest	fourOptionsDisplayed()	4 answer choices always visible
QuizScreenTest	selectOptionHighlightsIt()	Tapping an option highlights it blue

QuizScreenTest	submitButtonVisible()	Submit Answer button renders correctly
ResultsScreenTest	scorePercentageShown()	Final score percentage is displayed

5. Adaptive UI & Screen Sizes

The app is designed to adapt across different Android screen sizes and orientations using Jetpack Compose's adaptive layout system:

Screen Type	Width Breakpoint	Layout Adaptation
Compact (Phone)	< 600dp	Single column, full-width cards, bottom nav
Medium (Large phone / tablet)	600–840dp	2-column category grid, wider cards
Expanded (Tablet / foldable)	> 840dp	Side-by-side quiz layout, larger images
Landscape (Phone)	Height < 480dp	Horizontal scrolling, condensed header

6. Evaluation Rubric Self-Assessment

Category	Max pts	Implementation Evidence
Architecture Components	15	MVVM with ViewModel, StateFlow, Repository pattern, Hilt/manual DI
Navigation Components	10	NavHost, NavController, typed routes, back stack management
Activity Lifecycle	10	DisposableEffect for timer, onCleared(), config change survival
Testing	15	Unit (9 tests) + Navigation (6 tests) + UI Compose (8 tests)
Adaptive UI	15	WindowSizeClass breakpoints: Compact / Medium / Expanded
UI/UX	15	Mediterranean blue + warm earth tones, smooth animations, clean layout
Functionality	10	6 categories, timer, scoring, hints, progress tracking, results
Code Quality	10	Clean naming, KDoc comments, sealed classes, no hard-coded strings
TOTAL	100	

7. Known Issues & Future Improvements

- **Images are bundled locally:** All monument images are stored as drawable resources. A future version could load them from a remote CDN for easier content updates.
- **Modern Heritage & Natural Sites:** These two categories are marked 'Coming Soon' and do not have questions yet. They will be populated in a future release.
- **Timer on slow emulators:** On very slow emulators, the countdown timer may skip seconds. This does not occur on physical devices.
- **No user accounts:** Progress (streak, mastery) is stored locally and resets on app uninstall. Cloud sync is planned for a future version.

8. AI Usage Disclosure

In accordance with the exam requirements, the following is a full and transparent disclosure of all AI assistance used during the development of this project. All core logic, architecture decisions, and UI design were authored by the student; AI was used as a productivity and support tool.

Area	AI Tool	How It Was Used
Error Handling	Claude (Anthropic)	AI helped identify and fix runtime crashes and exception handling gaps — e.g. null safety
Adding New Categories	Claude (Anthropic)	AI was used to rapidly generate question content (question text, 4 options, correct answer)
Running & Writing Tests	Claude (Anthropic)	AI provided unit test, navigation test, and Compose UI test code templates and explained
App Icon Generation	Claude (Anthropic)	The app launcher icon (Roman column / heritage symbol) was generated with AI assistance
Project Report (PDF)	Claude (Anthropic)	This PDF report was generated using Claude based on the student's actual screenshots
README.md	Claude (Anthropic)	The GitHub README file was generated using Claude, incorporating the student's real p

Declaration: Jaouher Chouchane confirms that all AI assistance listed above was used as a learning and productivity tool. The core architecture design, implementation decisions, and understanding of the project are the student's own work. AI was not used to autonomously write the main application logic.